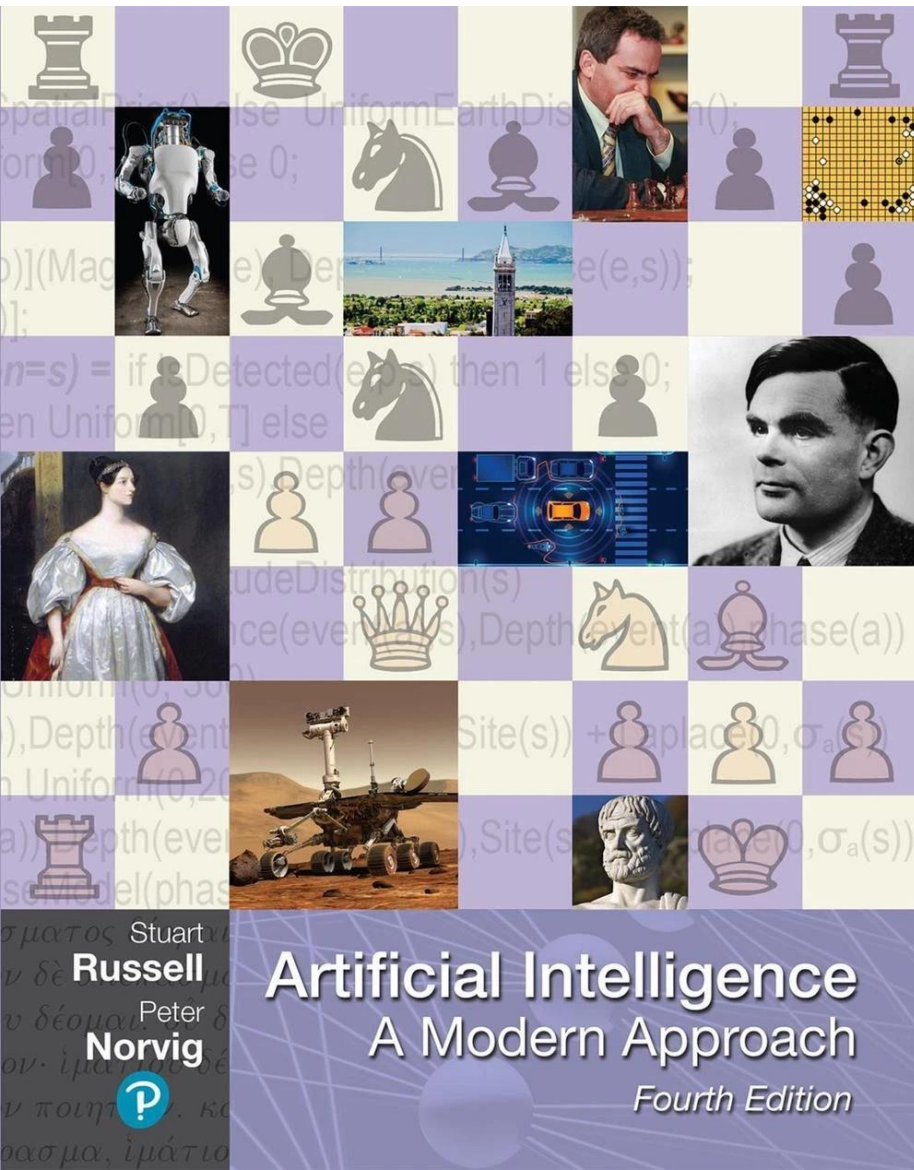


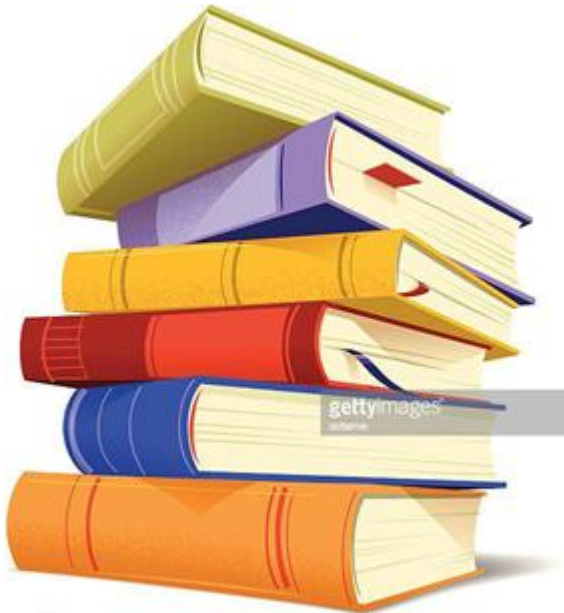
## Chapter 23

# Natural Language Processing



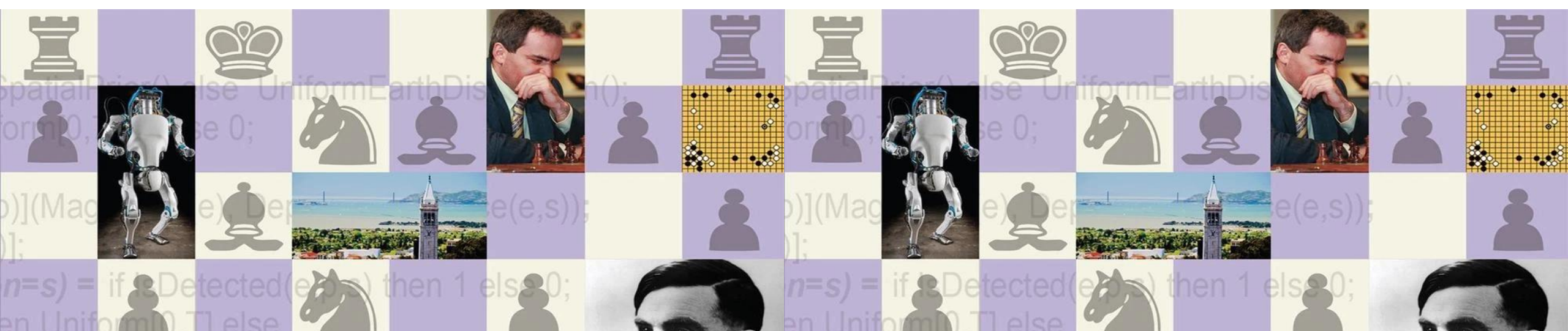
# Contents

---



- Language Models
- Grammar
- Parsing
- Augmented Grammars
- Complications of Real Natural Language
- Natural Language Tasks

# Language Models



# What a Language Model Does

---

- Assigns a **probability** to word sequences
- Predicts likely **next words** in context
- Supports correction, translation, and **question answering**
- Serves as a benchmark for **language understanding**

# Approximation Is Unavoidable

---

- **Natural languages** are not fully precise formal systems
- People share enough structure to communicate, not a perfect grammar
- Simple models can be **wrong but useful**
- The design question: what assumptions help the task?

# Bag-of-Words Model

---

- Represents text by **word counts or word presence**
- Ignores **word order** and syntactic structure
- Works well for topic or sentiment cues
- Fails when order changes meaning: “dog bites man” vs. “man bites dog”

# N-gram Word Models

---

- Use the previous **n - 1 words** to predict the next word
- Bigram:  $P(w_i \mid w_{i-1})$ ; trigram:  $P(w_i \mid w_{i-2}, w_{i-1})$
- Capture local patterns such as **collocations**
- Still miss long-range syntax and meaning

# Character N-grams and Robustness

---

- **Character-level** models predict letters or characters
- Useful for spelling correction and unknown words
- Can model morphology: prefixes, suffixes, endings
- Need longer contexts to capture word-level meaning



# Smoothing N-gram Models

---

- Raw counts make unseen n-grams look **impossible**
- **Smoothing** reserves probability mass for rare events
- Backoff uses shorter contexts when long contexts are sparse
- Preprocessing reduces noise in huge feature spaces

# Word Representations

---

- N-grams treat words mostly as **symbols**
- Richer representations capture similarity among words
- Useful generalization: “cat” and “dog” behave similarly in many contexts
- Chapter 24 continues with **word embeddings** and deep learning

# Part-of-Speech Tagging

---

- **POS tags** label words as noun, verb, adjective, and so on
- Tags support generalizations about syntax
- Ambiguous words need context: “record” as noun or verb
- Tagging is a useful first step for many NLP tasks

| Tag  | Word           | Description              | Tag   | Word            | Description           |
|------|----------------|--------------------------|-------|-----------------|-----------------------|
| CC   | <i>and</i>     | Coordinating conjunction | PRP\$ | <i>your</i>     | Possessive pronoun    |
| CD   | <i>three</i>   | Cardinal number          | RB    | <i>quickly</i>  | Adverb                |
| DT   | <i>the</i>     | Determiner               | RBR   | <i>quicker</i>  | Adverb, comparative   |
| EX   | <i>there</i>   | Existential there        | RBS   | <i>quickest</i> | Adverb, superlative   |
| FW   | <i>per se</i>  | Foreign word             | RP    | <i>off</i>      | Particle              |
| IN   | <i>of</i>      | Preposition              | SYM   | <i>+</i>        | Symbol                |
| JJ   | <i>purple</i>  | Adjective                | TO    | <i>to</i>       | to                    |
| JJR  | <i>better</i>  | Adjective, comparative   | UH    | <i>eureka</i>   | Interjection          |
| JJS  | <i>best</i>    | Adjective, superlative   | VB    | <i>talk</i>     | Verb, base form       |
| LS   | <i>I</i>       | List item marker         | VBD   | <i>talked</i>   | Verb, past tense      |
| MD   | <i>should</i>  | Modal                    | VBG   | <i>talking</i>  | Verb, gerund          |
| NN   | <i>kitten</i>  | Noun, singular or mass   | VCN   | <i>talked</i>   | Verb, past participle |
| NNS  | <i>kittens</i> | Noun, plural             | VBP   | <i>talk</i>     | Verb, non-3rd-sing    |
| NNP  | <i>Ali</i>     | Proper noun, singular    | VBZ   | <i>talks</i>    | Verb, 3rd-sing        |
| NNPS | <i>Fords</i>   | Proper noun, plural      | WDT   | <i>which</i>    | Wh-determiner         |
| PDT  | <i>all</i>     | Predeterminer            | WP    | <i>who</i>      | Wh-pronoun            |
| POS  | <i>'s</i>      | Possessive ending        | WP\$  | <i>whose</i>    | Possessive wh-pronoun |
| PRP  | <i>you</i>     | Personal pronoun         | WRB   | <i>where</i>    | Wh-adverb             |
| \$   | <i>\$</i>      | Dollar sign              | #     | <i>#</i>        | Pound sign            |
| “    | <i>‘</i>       | Left quote               | ”     | <i>’</i>        | Right quote           |
| (    | <i>[</i>       | Left parenthesis         | )     | <i>]</i>        | Right parenthesis     |
| ,    | <i>,</i>       | Comma                    | .     | <i>!</i>        | Sentence end          |
| :    | <i>;</i>       | Mid-sentence punctuation |       |                 |                       |

Part-of-speech tags (with an example word for each tag) for the Penn Treebank corpus (Marcus *et al.*, 1993). Here “3rd-sing” is an abbreviation for “third person singular present tense.”

# Comparing Language Models

---

- Unigrams are fluent only at the level of word frequency
- Higher n-grams improve **local coherence**
- Large n-grams can memorize rather than generalize
- Better models combine richer **words, context, and structure**

# Grammar



# Grammar and Language

---

- A **grammar** defines allowable phrase structures
- A **language** is the set of sentences generated by the grammar
- Natural-language grammars are approximate and domain-limited
- AIMA uses a tiny Wumpus-world language,  $E_0$

# Probabilistic Context-Free Grammar

---

- **PCFG** attaches probabilities to grammar rules
- **Context-free**: a rule can be used in any context
- The probability of a parse tree multiplies rule probabilities
- Multiple parses can be ranked by probability



Figure 23.2

---

|             |                         |        |                                   |
|-------------|-------------------------|--------|-----------------------------------|
| $S$         | $\rightarrow NP VP$     | [0.90] | I + feel a breeze                 |
|             | $S Conj S$              | [0.10] | I feel a breeze + and + It stinks |
| $NP$        | $\rightarrow Pronoun$   | [0.25] | I                                 |
|             | $Name$                  | [0.10] | Ali                               |
|             | $Noun$                  | [0.10] | pits                              |
|             | $Article Noun$          | [0.25] | the + wumpus                      |
|             | $Article Adjs Noun$     | [0.05] | the + smelly dead + wumpus        |
|             | $Digit Digit$           | [0.05] | 3 4                               |
|             | $NP PP$                 | [0.10] | the wumpus + in 1 3               |
|             | $NP RelClause$          | [0.05] | the wumpus + that is smelly       |
|             | $NP Conj NP$            | [0.05] | the wumpus + and + I              |
| $VP$        | $\rightarrow Verb$      | [0.40] | stinks                            |
|             | $VP NP$                 | [0.35] | feel + a breeze                   |
|             | $VP Adjective$          | [0.05] | smells + dead                     |
|             | $VP PP$                 | [0.10] | is + in 1 3                       |
|             | $VP Adverb$             | [0.10] | go + ahead                        |
| $Adjs$      | $\rightarrow Adjective$ | [0.80] | smelly                            |
|             | $Adjective Adjs$        | [0.20] | smelly + dead                     |
| $PP$        | $\rightarrow Prep NP$   | [1.00] | to + the east                     |
| $RelClause$ | $\rightarrow RelPro VP$ | [1.00] | that + is smelly                  |

---

The grammar for  $E_0$ , with example phrases for each rule. The syntactic categories are sentence ( $S$ ), noun phrase ( $NP$ ), verb phrase ( $VP$ ), list of adjectives ( $Adjs$ ), prepositional phrase ( $PP$ ), and relative clause ( $RelClause$ ).

---

# Overgeneration and Undergeneration

---

- **Overgeneration:** grammar accepts bad sentences
- **Undergeneration:** grammar rejects good sentences
- Tiny grammars reveal the tradeoff clearly
- Better coverage requires richer rules or learned grammars

# Lexicon: Words Meet Grammar

---

- The **lexicon** maps words to lexical categories
- Open classes: nouns, verbs, adjectives can grow
- Closed classes: articles, pronouns, prepositions are smaller
- Lexical probabilities help choose likely words

|                  |   |                                                                                               |
|------------------|---|-----------------------------------------------------------------------------------------------|
| <i>Noun</i>      | → | <b>stench</b> [0.05]   <b>breeze</b> [0.10]   <b>wumpus</b> [0.15]   <b>pits</b> [0.05]   ... |
| <i>Verb</i>      | → | <b>is</b> [0.10]   <b>feel</b> [0.10]   <b>smells</b> [0.10]   <b>stinks</b> [0.05]   ...     |
| <i>Adjective</i> | → | <b>right</b> [0.10]   <b>dead</b> [0.05]   <b>smelly</b> [0.02]   <b>breezy</b> [0.02] ...    |
| <i>Adverb</i>    | → | <b>here</b> [0.05]   <b>ahead</b> [0.05]   <b>nearby</b> [0.02]   ...                         |
| <i>Pronoun</i>   | → | <b>me</b> [0.10]   <b>you</b> [0.03]   <b>I</b> [0.10]   <b>it</b> [0.10]   ...               |
| <i>RelPro</i>    | → | <b>that</b> [0.40]   <b>which</b> [0.15]   <b>who</b> [0.20]   <b>whom</b> [0.02]   ...       |
| <i>Name</i>      | → | <b>Ali</b> [0.01]   <b>Bo</b> [0.01]   <b>Boston</b> [0.01]   ...                             |
| <i>Article</i>   | → | <b>the</b> [0.40]   <b>a</b> [0.30]   <b>an</b> [0.10]   <b>every</b> [0.05]   ...            |
| <i>Prep</i>      | → | <b>to</b> [0.20]   <b>in</b> [0.10]   <b>on</b> [0.05]   <b>near</b> [0.10]   ...             |
| <i>Conj</i>      | → | <b>and</b> [0.50]   <b>or</b> [0.10]   <b>but</b> [0.20]   <b>yet</b> [0.02]   ...            |
| <i>Digit</i>     | → | <b>0</b> [0.20]   <b>1</b> [0.20]   <b>2</b> [0.20]   <b>3</b> [0.20]   <b>4</b> [0.20]   ... |

The lexicon for  $E_0$ . *RelPro* is short for relative pronoun, *Prep* for preposition, and *Conj* for conjunction. The sum of the probabilities for each category is 1.

---

# Parsing



# Parsing as Search

---

- **Parsing** uncovers phrase structure for a word string
- Top-down: start from **S** and expand rules
- Bottom-up: start from words and combine constituents
- A parse tree explains how the sentence is generated

Figure 23.4

| List of items                               | Rule                            |
|---------------------------------------------|---------------------------------|
| <i>S</i>                                    |                                 |
| <i>NP VP</i>                                | <i>S</i> → <i>NP VP</i>         |
| <i>NP VP Adjective</i>                      | <i>VP</i> → <i>VP Adjective</i> |
| <i>NP Verb Adjective</i>                    | <i>VP</i> → <i>Verb</i>         |
| <i>NP Verb <b>dead</b></i>                  | <i>Adjective</i> → <b>dead</b>  |
| <i>NP <b>is dead</b></i>                    | <i>Verb</i> → <b>is</b>         |
| <i>Article Noun <b>is dead</b></i>          | <i>NP</i> → <i>Article Noun</i> |
| <i>Article <b>wumpus</b> <b>is dead</b></i> | <i>Noun</i> → <b>wumpus</b>     |
| <i><b>the wumpus is dead</b></i>            | <i>Article</i> → <b>the</b>     |

Parsing the string “The wumpus is dead” as a sentence, according to the grammar  $E_0$ . Viewed as a top-down parse, we start with  $S$ , and on each step match one nonterminal  $X$  with a rule of the form  $(X \rightarrow Y \dots)$  and replace  $X$  in the list of items with  $Y \dots$ ; for example replacing  $S$  with the sequence  $NP VP$ . Viewed as a bottom-up parse, we start with the words “the wumpus is dead”, and on each step match a string of tokens such as  $(Y \dots)$  against a rule of the form  $(X \rightarrow Y \dots)$  and replace the tokens with  $X$ ; for example replacing “the” with  $Article$  or  $Article Noun$  with  $NP$ .

# Chart Parsing

---

- A **chart** stores partial parse results
- Shared subphrases are computed once and reused
- Useful because many parses contain overlapping spans
- Dynamic programming avoids repeated dead-end work



# CYK Algorithm

---

- A bottom-up **chart parser** for PCFGs
- Requires grammar in **Chomsky Normal Form**
- Finds the most probable tree for every span
- Guarantees optimal parse under the grammar

**function** CYK-PARSE(*words*, *grammar*) **returns** a table of parse trees

**inputs:** *words*, a list of words

*grammar*, a structure with LEXICALRULES and GRAMMARRULES

$T \leftarrow$  a table      //  $T[X, i, k]$  is most probable  $X$  tree spanning  $words_{i:k}$

$P \leftarrow$  a table, initially all 0      //  $P[X, i, k]$  is probability of tree  $T[X, i, k]$

// Insert lexical categories for each word.

**for**  $i = 1$  **to** LEN(*words*) **do**

**for each** ( $X, p$ ) **in** *grammar*.LEXICALRULES( $words_i$ ) **do**

$P[X, i, i] \leftarrow p$

$T[X, i, i] \leftarrow \text{TREE}(X, words_i)$

// Construct  $X_{i:k}$  from  $Y_{i:j} + Z_{j+1:k}$ , shortest spans first.

**for each** ( $i, j, k$ ) **in** SUBSPANS(LEN(*words*)) **do**

**for each** ( $X, Y, Z, p$ ) **in** *grammar*.GRAMMARRULES **do**

$PYZ \leftarrow P[Y, i, j] \times P[Z, j+1, k] \times p$

**if**  $PYZ > P[X, i, k]$  **do**

$P[X, i, k] \leftarrow PYZ$

$T[X, i, k] \leftarrow \text{TREE}(X, T[Y, i, j], T[Z, j+1, k])$

**return**  $T$

**function** SUBSPANS( $N$ ) **yields** ( $i, j, k$ ) tuples

**for**  $length = 2$  **to**  $N$  **do**

**for**  $i = 1$  **to**  $N + 1 - length$  **do**

# Probability and Complexity

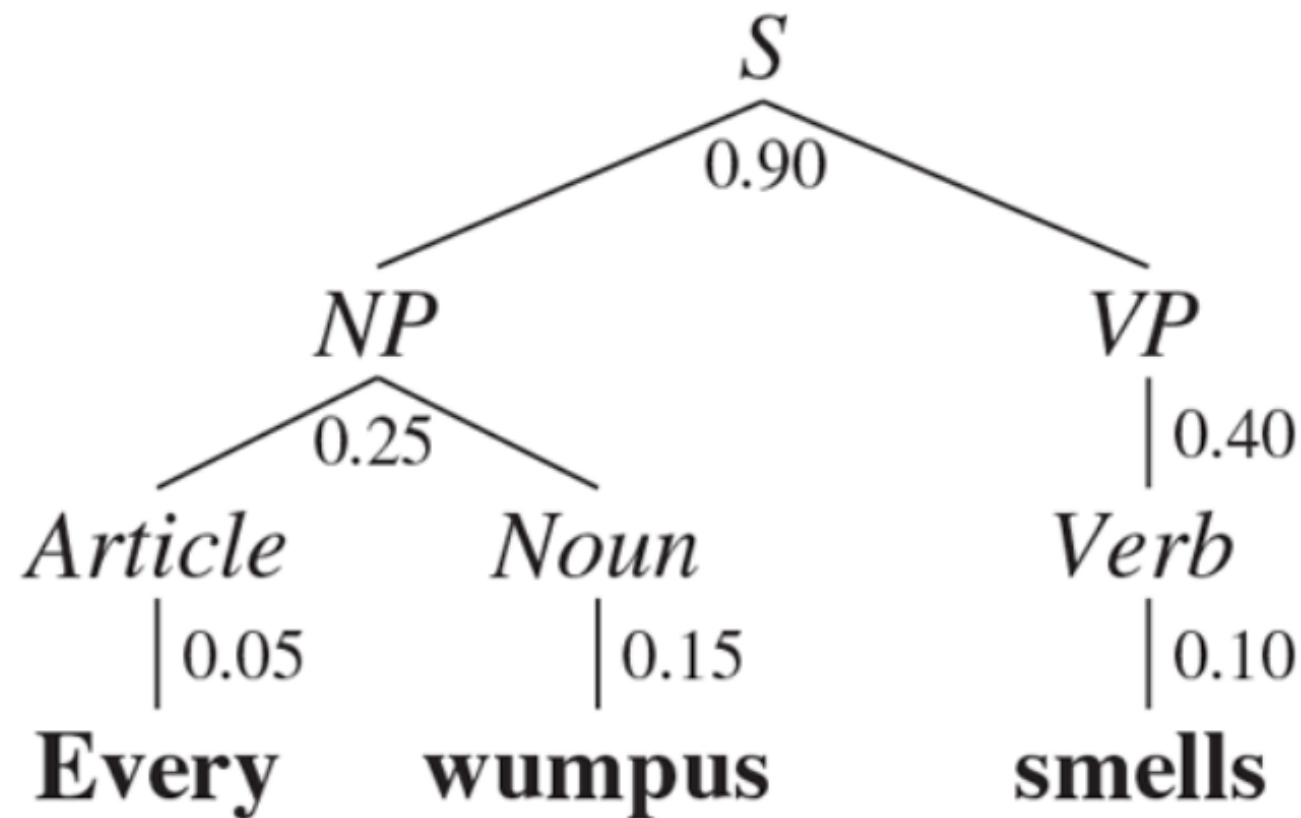
---

- Tree probability multiplies the selected rule probabilities
- CYK space:  **$O(n^2m)$** ; time:  **$O(n^3m)$**
- $n$  = sentence length;  $m$  = nonterminal count
- Natural-language parsers often use faster approximate search

---

Figure 23.6

---



Parse tree for the sentence “Every wumpus smells” according to the grammar  $E_0$ . Each interior node of the tree is labeled with its probability. The probability of the tree as a whole is  $0.9 \times 0.25 \times 0.05 \times 0.15 \times 0.40 \times 0.10 = 0.0000675$ . The tree can also be written in linear form as  $[S[NP[Article \textbf{every}][Noun \textbf{wumpus}]] [VP[Verb \textbf{smells}]]]$ .

---

# Faster Parsing Strategies

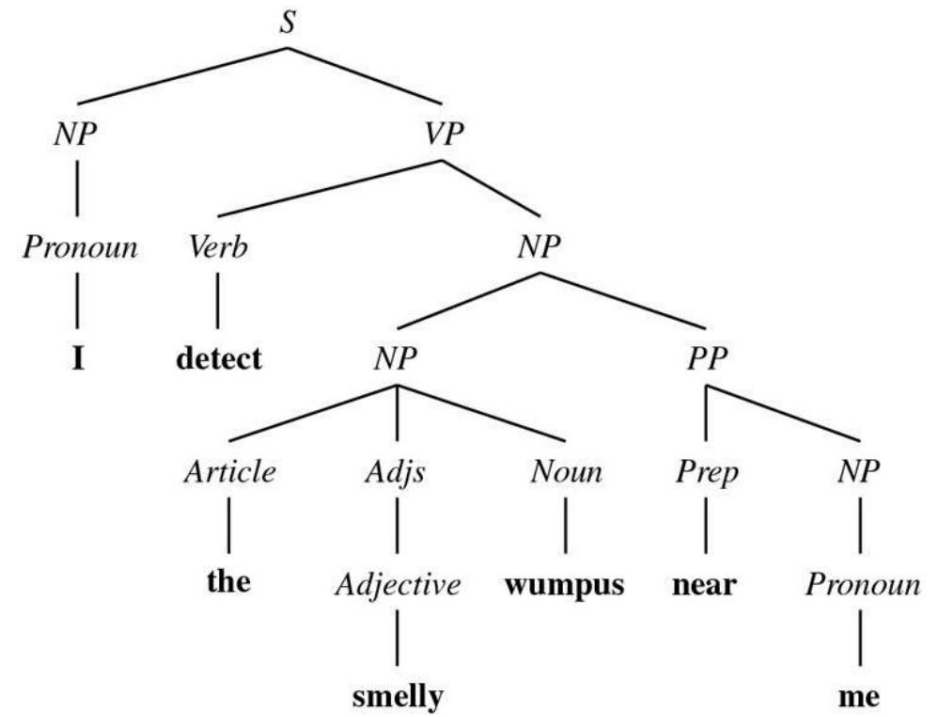
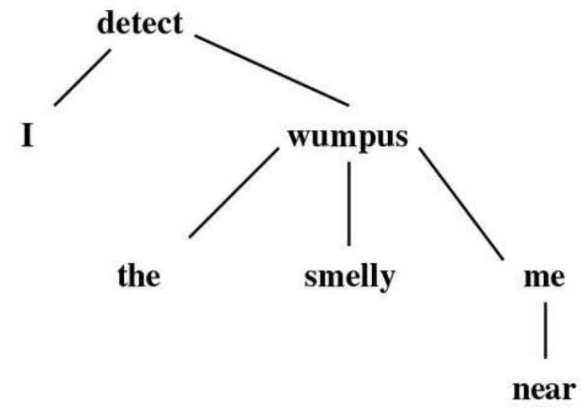
---

- **A\*** search uses heuristics to avoid full enumeration
- **Beam search** keeps only the  $b$  best partial parses
- $b = 1$  gives a deterministic parser
- **Shift-reduce** parsing chooses shift or reduce actions left to right

# Dependency Parsing

---

- **Dependency grammar** links words directly
- Focuses on head-dependent relations
- Does not require phrase constituents such as NP or VP
- Often convenient for downstream semantic tasks



A dependency-style parse (top) and the corresponding phrase structure parse (bottom) for the sentence *I detect the smelly wumpus near me.*

# Learning Parsers from Treebanks

---

- Writing a broad grammar by hand is difficult
- A **treebank** provides sentences with parse trees
- PCFG rule probabilities can be estimated by counting
- Smoothing is still needed for rare structures



```

[ [S [NP-2 Her eyes]
  [VP were
    [VP glazed
      [NP *-2]
      [SBAR-ADV as if
        [S [NP she]
          [VP did n't
            [VP [VP hear [NP *-1]]
              or
              [VP [ADVP even] see [NP *-1]]
              [NP-1 him]]]]]]]]
    ]
  ]
]

```

Annotated tree for the sentence “Her eyes were glazed as if she didn’t hear or even see him.” from the Penn Treebank. Note a grammatical phenomenon we have not covered yet: the movement of a phrase from one part of the tree to another. This tree analyzes the phrase “hear or even see him” as consisting of two constituent *VP*s, *[VP hear [NP \*-1]]* and *[VP [ADVP even] see [NP \*-1]]*, both of which have a missing object, denoted \*-1, which refers to the *NP* labeled elsewhere in the tree as *[NP-1 him]*. Similarly, the *[NP \*-2]* refers to the *[NP-2 Her eyes]*.

---

# Supervised and Beyond

---

- **Supervised parsing** learns from sentence-tree pairs
- **Unsupervised parsing** learns from sentences without trees
- **Semisupervised parsing** combines small treebanks with large raw corpora
- Partial bracketing can provide weak structural clues

# Augmented Grammars



# Why Augment a Grammar?

---

- Plain CFG categories are too coarse
- “I ate a banana” vs. “Me ate a banana”
- Agreement, case, and head words affect grammaticality
- **Augmented categories** carry structured information

# Lexicalized PCFGs

---

- A **lexicalized PCFG** conditions rule probabilities on head words
- Captures preferences like “ate a banana” over “ate a bandanna”
- Backoff is needed because word-pair tables are huge
- Bridges n-gram evidence and hierarchical grammar

# Agreement and Feature Passing

---

- $NP(c, pn, n)$ : case, person-number, head noun
- $VP(pn, v)$ : agreement features and head verb
- Rules can require **matching features**
- Invalid combinations receive low or zero probability

$$\begin{aligned}
S(v) &\rightarrow NP(Sbj, pn, n) VP(pn, v) \mid \dots \\
NP(c, pn, n) &\rightarrow Pronoun(c, pn, n) \mid Noun(c, pn, n) \mid \dots \\
VP(pn, v) &\rightarrow Verb(pn, v) NP(Obj, pn, n) \mid \dots \\
PP(head) &\rightarrow Prep(head) NP(Obj, pn, h) \\
Pronoun(Sbj, 1S, \mathbf{I}) &\rightarrow \mathbf{I} \\
Pronoun(Sbj, 1P, \mathbf{we}) &\rightarrow \mathbf{we} \\
Pronoun(Obj, 1S, \mathbf{me}) &\rightarrow \mathbf{me} \\
Pronoun(Obj, 3P, \mathbf{them}) &\rightarrow \mathbf{them} \\
Verb(3S, \mathbf{see}) &\rightarrow \mathbf{see}
\end{aligned}$$

Part of an augmented grammar that handles case agreement, subject–verb agreement, and head words. Capitalized names are constants: *Sbj*, and *Obj*, for subjective and objective case; *1S* for first person singular; *1P* and *3P* for first and third person plural. As usual, lowercase names are variables. For simplicity, the probabilities have been omitted.

---

# Semantic Interpretation

---

- Augment grammar rules with **meaning representations**
- Use **compositional semantics**: phrase meaning from subphrase meanings
- Arithmetic expressions provide a clear toy example
- The same idea extends to natural-language predicates



$$Exp(op(x_1, x_2)) \rightarrow Exp(x_1) Operator(op) Exp(x_2)$$

$$Exp(x) \rightarrow ( Exp(x) )$$

$$Exp(x) \rightarrow Number(x)$$

$$Number(x) \rightarrow Digit(x)$$

$$Number(10 \times x_1 + x_2) \rightarrow Number(x_1) Digit(x_2)$$

$$Operator(+ ) \rightarrow +$$

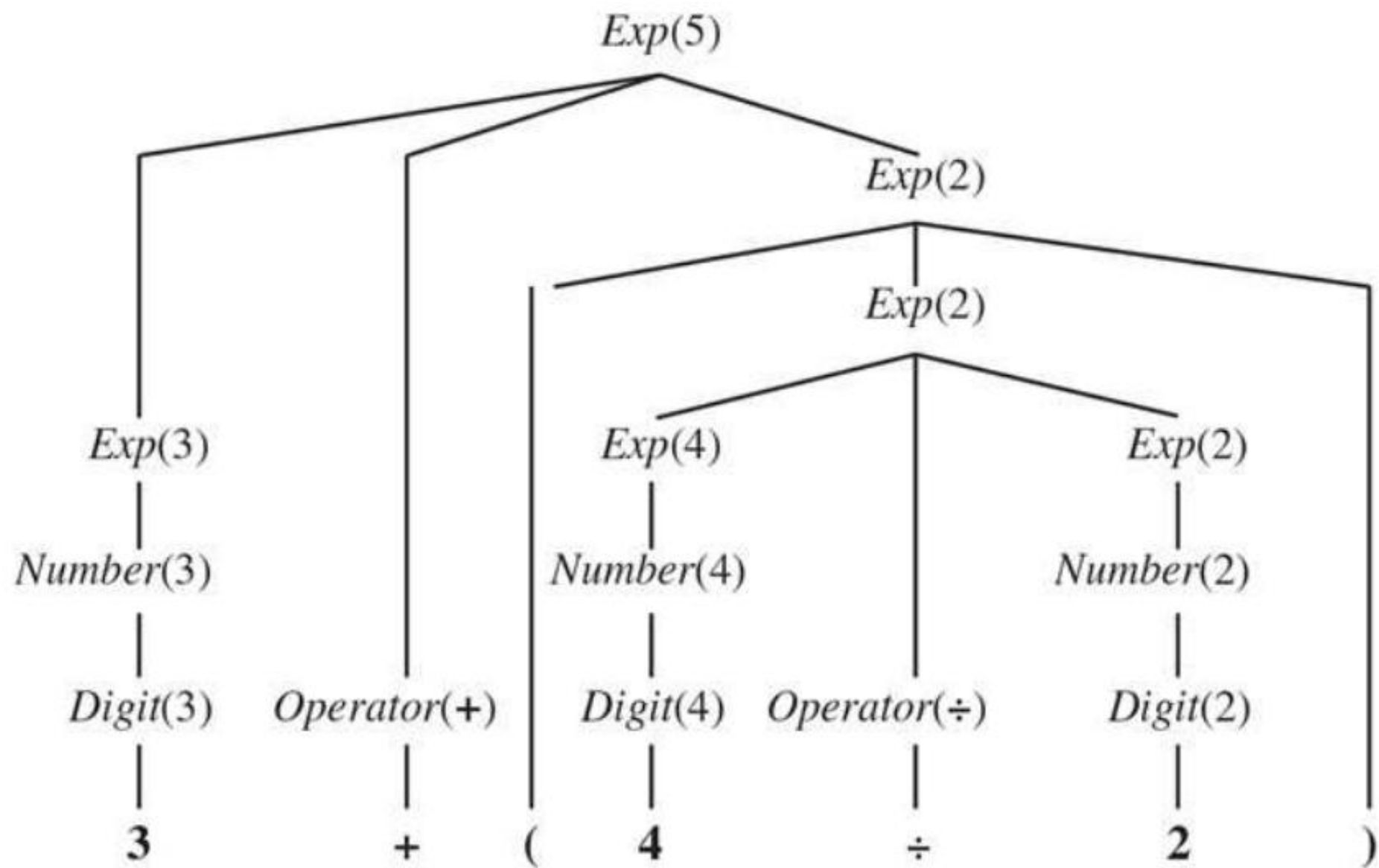
$$Operator(- ) \rightarrow -$$

$$Operator(\times) \rightarrow \times$$

$$Operator(\div) \rightarrow \div$$

$$Digit(0) \rightarrow \mathbf{0}$$

$$Digit(1) \rightarrow \mathbf{1}$$



Parse tree with semantic interpretations for the string "3 + (4 ÷ 2)".

# Lambda Semantics for Sentences

---

- Names denote individuals: Ali, Bo
- Verbs can denote **lambda expressions**
- “loves” becomes a relation waiting for arguments
- Parsing can produce a logical form such as Loves(Ali, Bo)

Figure 23.12

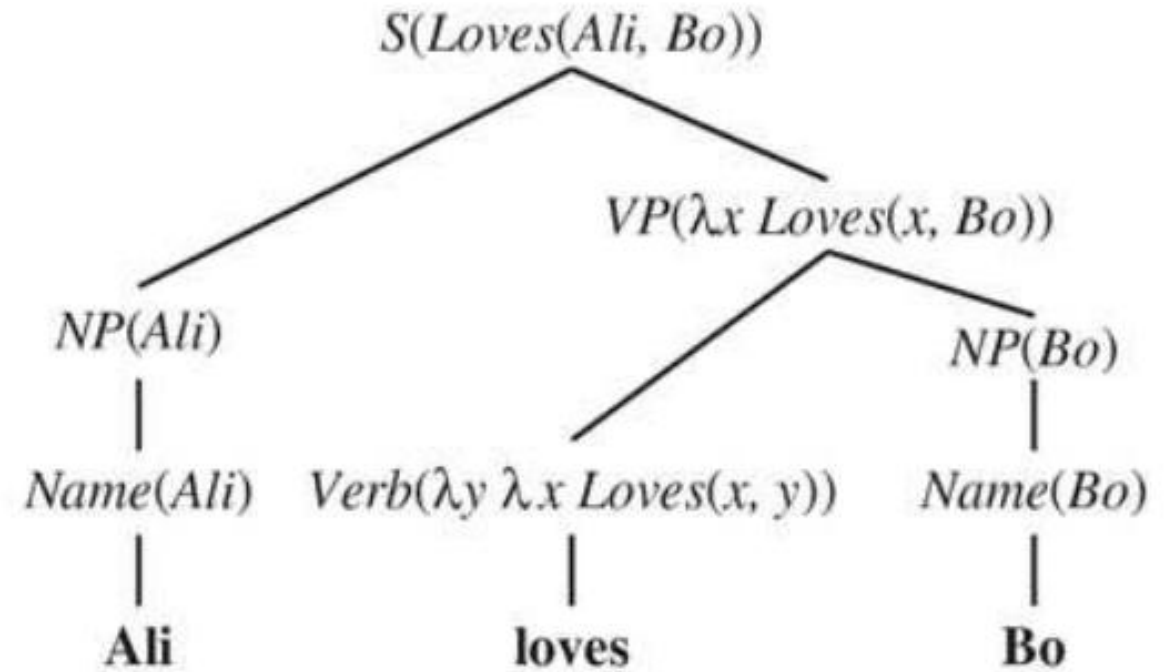
$S(pred(n)) \rightarrow NP(n) VP(pred)$   
 $VP(pred(n)) \rightarrow Verb(pred) NP(n)$   
 $NP(n) \rightarrow Name(n)$

$Name(Ali) \rightarrow \mathbf{Ali}$

$Name(Bo) \rightarrow \mathbf{Bo}$

$Verb(\lambda y \lambda x Loves(x, y)) \rightarrow \mathbf{loves}$

(a)



(b)

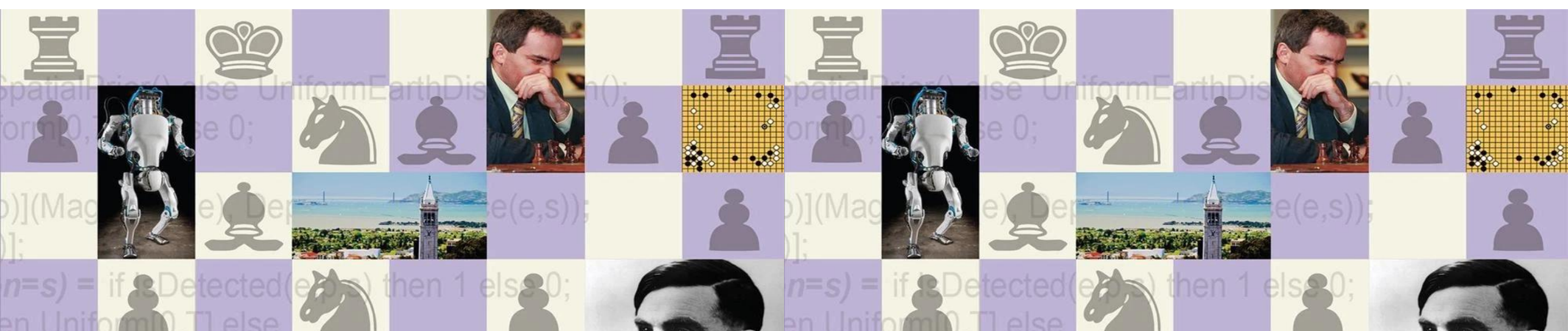
(a) A grammar that can derive a parse tree and semantic interpretation for “Ali loves Bo” (and three other sentences). Each category is augmented with a single argument representing the semantics. (b) A parse tree with semantic interpretations for the string “Ali loves Bo.”

# Learning Semantic Grammars

---

- Treebanks usually contain syntax, not semantics
- Semantic grammars need sentence-meaning examples
- Training pairs may use logical forms or answers
- Goal: parse language into **executable meaning**

# Complications of Real Natural Language



# Why Real Language Is Hard

---

- Real English is far richer than E<sub>0</sub>
- Formal grammars must handle **scope, context, movement, tense, ambiguity**
- Many phenomena require knowledge beyond grammar
- Robust systems combine multiple models

# Quantification and Scope

---

- Quantifiers such as **every**, **some**, and **a** introduce scope
- Different scopes can yield different meanings
- A grammar may produce a **quasi-logical form**
- Separate algorithms choose preferred scope readings



# Pragmatics and Speech Acts

---

- **Pragmatics** fills in context-dependent meaning
- Pronouns, deixis, and ellipsis need the current situation
- Utterances can be questions, commands, promises, or warnings
- The same sentence form may have different communicative functions

# Long-Distance Dependencies and Tense

---

- A phrase can be interpreted far from where it appears
- Gaps and movement complicate local parsing
- Tense links events to time and aspect
- Small grammar changes can expose large linguistic complexity

# Ambiguity and Metonymy

---

- **Ambiguity** is normal, not exceptional
- Syntactic, semantic, and pragmatic ambiguity interact
- **Metonymy** lets one entity stand for a related one
- Disambiguation needs grammar, language statistics, context, and world knowledge



# Task Families in NLP

---

- NLP tasks transform, extract, retrieve, or generate language
- Language models provide **probability estimates**
- Grammars and parsers provide **structure**
- Semantics connects text to actions, answers, or databases

# Information Extraction

---

- **Information extraction** finds structured facts in text
- Typical targets: entities, relations, events, dates
- Parsing helps identify who did what to whom
- Useful for building knowledge bases from documents

# Machine Translation and Speech

---

- **Machine translation** chooses a target sentence that preserves meaning
- Language models help select fluent translations
- **Speech recognition** maps acoustic signals to word sequences
- Both tasks combine uncertain evidence with linguistic constraints

# Question Answering

---

- **Question answering** returns an answer, not just documents
- Parsing can reveal the expected answer type
- Information retrieval supplies candidate evidence
- Semantic interpretation links questions to facts or databases



# Summary

---

- **N-gram models** are simple, useful, and data-sparse
- **PCFGs** model hierarchical phrase structure
- **Parsing** searches for likely structures; CYK is exact but costly
- **Augmented grammars** add agreement, lexical heads, and semantics
- Real NLP requires handling ambiguity, context, and task goals

